Enterprise Microservices Architecture & Service Decomposition Document

1️⃣ SYSTEM OVERVIEW

High-level Architecture Summary

The current system is a monolithic Spring Boot application (`PlaygroundService_api`) that orchestrates multiple AI and integration capabilities, including deployable workflows, tool execution, model/agent management, and external MCP (Model Context Protocol) integrations. While cohesive, the increasing complexity of deployment orchestration, asynchronous workflow executions, and integration with third-party tool sets (like D365) signals the need for a microservice decomposition.

Current Project Analysis Summary

The existing codebase contains over 40 distinct services in a single monolithic boundary. Key responsibilities currently entangled include:

- Managing AI agents, models, and data stores.
- Handling complex, long-running deployments (e.g., deploying models, syncing vector DBs, linking external endpoints).
- Executing workflows via asynchronous managers (`WorkflowAsyncExecutor`) and real-time interaction modules (`ChatController`).
- Communicating with external MCP servers and HTTP-based tools.

Identified Bounded Contexts

1. **Deployment Operations Context**: Managing the lifecycle, provisioning, and status tracking of infrastructure/models.
2. **Execution Context**: Running steps, handling chat memory, and advancing workflows.
3. **Integration Context**: Executing external tools and MCP endpoints securely.
4. **Agent/Model Core Context**: Managing metadata, versions, and configurations of AI components.
5. **Project Management Context**: Ownership, workspace definition, and RBAC.

Suggested Microservice Separation Strategy

Adopt a **Domain-Driven Design (DDD)** approach to extract the highly coupled orchestration engines (`Deployment` and `Workflow Execution`) from the standard CRUD resources (`Entities`). We will use an API Gateway to route traffic, an Event Bus (Kafka) for cross-service asynchronous communication, and dedicated databases per microservice to enforce pure bounded contexts.

Monolith vs Microservice Comparison

Aspect	Current Monolith	Proposed Microservices
Scalability	Hard to scale execution independently of simple CRUD operations.	Execution nodes and deployment workers scale horizontally.
Fault Tolerance	A failure in deployment orchestration crashes the execution engine.	Isolated failures; circuit breakers prevent cascading failures.

Database	Single shared database risks locking and tight coupling.	Polyglot persistence (e.g., NoSQL for active executions, SQL for metadata).

2️⃣ IDENTIFIED MICROSERVICES

1. **Deployment Service**
    - **Purpose**: Provisions infrastructure, configures vector databases, deploys AI agents/models.
    - **Why Isolated**: Highly stateful, interacts heavily with external cloud providers (Azure/AWS), requires long-running polling/callbacks.
    - **Trigger**: When a workflow is published or a new model is onboarded.
    - **Interaction**: Emits `DeploymentStatusEvent` to Kafka; consumed by the Gateway/Execution Service.
    - **Database Strategy**: Relational SQL (PostgreSQL) for ACID compliance on deployment states.
2. **Workflow Execution Service**
    - **Purpose**: Runs the actual AI workflows, manages chat sessions, handles retries state securely.
    - **Why Isolated**: Resource-intensive (CPU/Memory). Needs specialized parallel execution scaling.
    - **Trigger**: User starts a chat or triggers an API workflow execution.
    - **Interaction**: Synchronous API calls to Tool Service; Asynchronous events for status updates.
    - **Database Strategy**: Document DB (MongoDB) for unstructured flow JSONs and chat histories. Redis for session cache.
3. **Tool Execution Service**
    - **Purpose**: Acts as an egress proxy executing 3rd-party OpenAPI tools, handling OAuth flows for tools.
    - **Why Isolated**: High likelihood of blocking I/O and timeouts. Rate-limiting logic belongs here.
4. **MCP Integration Service**
    - **Purpose**: Manages Model Context Protocol (JSON-RPC) standard communication.
5. **Project Management Service**
    - **Purpose**: Handles workspaces, RBAC, ownership, and collaborative sharing.
6. **Agent Management Service**
    - **Purpose**: Core catalog for AI Agents, Prompts, Models, Datasets, and their versioning.
7. **Cache Service**
    - **Purpose**: Redis-based ephemeral storage for rate limiting, active WebSocket sessions, and fast lookups.
8. **Status/Monitoring Service**
    - **Purpose**: Aggregates distributed tracing, logging, and act as a WebSocket server for UI status updates.
9. **Authentication Service**
    - **Purpose**: Validates JWTs, manages OAuth2 flows, handles API key generation.
10. **API Gateway**

- **Purpose**: Spring Cloud Gateway for routing, centralized auth termination, and rate limiting.

3️⃣ DEEP DIVE: DEPLOYMENT SERVICE

Deployment Lifecycle

Deployments move through a strictly controlled state machine: `CREATED -> IN_PROGRESS -> SUCCESS / FAILED`. External integrations (like Azure DevOps) are triggered, and the service either polls for completion or waits for a webhook (`/callback`).

A) Entity Layer

- **Deployment**
  - `id` (UUID): Primary Key.
  - `workflowId` , `userId` : Foreign links.
  - `status` (Enum): Tracking overall state.
  - `createdAt` , `updatedAt` : Audit fields for tracing lifecycle duration.
- **DeploymentTask**
  - Represents individual sub-tasks (e.g., provision Vector DB, deploy LLM wrapper).
  - Maintains a `taskStatus` to track granular progression.
- **DeploymentCallbackDTO**: Used to ingest external webhooks.

B) Repository Layer

- `DeploymentRepository` (JpaRepository): Optimized queries like `findByWorkflowIdAndStatus`.
- **Custom Queries**: Uses `@Query` or Specifications to effectively fetch active deployments without fetching heavy unneeded relational data.

C) Service Layer

- **Core Methods**: `deployWorkflow()` , `handleCallback()` , `updateTaskStatus()` .
- **Transaction Boundaries**: Heavy tasks run in `@Async` blocks outside the main DB transaction to prevent connection pooling exhaustion.
- **Retry & Idempotency**: Relying on unique task correlation IDs to prevent duplicate deployments.

D) Controller Layer (15 APIs)

```java
@RestController
@RequestMapping("/api/deployments")
public class DeploymentController {

    // 1. Initiate a new deployment orchestration
    // Request: { "workflowId": "..." }
    @PostMapping
    public ResponseEntity<List<DeployApiResponseDTO>> createDeployment(@RequestParam String
workflowId);

    // 2. Fetch overall deployment status
    // Response: { "status": "IN_PROGRESS", "tasks": [...] }
    @GetMapping("/{id}")
    public ResponseEntity<DeploymentDTO> getDeploymentById(@PathVariable UUID id);
```

```java
    // 3. Filter active or failed deployments
    @GetMapping("/status/{status}")
    public ResponseEntity<List<DeploymentDTO>> getDeploymentsByStatus(@PathVariable String
status);

    // 4. Cancel a hung deployment
    @PostMapping("/{id}/cancel")
    public ResponseEntity<Void> cancelDeployment(@PathVariable UUID id);

    // 5. Retry a failed deployment task
    @PostMapping("/{id}/retry")
    public ResponseEntity<DeploymentDTO> retryDeployment(@PathVariable UUID id);

    // 6. Get all deployments for a specific project
    @GetMapping("/project/{projectId}")
    public ResponseEntity<List<DeploymentDTO>> getProjectDeployments(@PathVariable UUID
projectId);

    // 7. EXTERNAL Webhook callback handler
    @PostMapping("/callback")
    public ResponseEntity<Void> handleCallback(@RequestBody DeploymentCallbackDTO callback);

    // 8. Historical execution logs for deployments
    @GetMapping("/history/{projectId}")
    public ResponseEntity<List<DeploymentHistoryDTO>> getDeploymentHistory(@PathVariable
UUID projectId);

    // 9. Soft-delete a deployment record
    @DeleteMapping("/{id}")
    public ResponseEntity<Void> deleteDeployment(@PathVariable UUID id);

    // 10. Bulk trigger deployments deployment
    @PostMapping("/bulk")
    public ResponseEntity<List<DeployApiResponseDTO>> bulkDeploy(@RequestBody
BulkDeployRequest req);

    // 11. Fetch metrics (latency, success rate)
    @GetMapping("/metrics")
    public ResponseEntity<DeploymentMetricsDTO> getMetrics();

    // 12. Fetch raw logs of a specific deployment
    @GetMapping("/logs/{id}")
    public ResponseEntity<String> getDeploymentLogs(@PathVariable UUID id);

    // 13. Pause an active multi-stage deployment
    @PutMapping("/{id}/pause")
    public ResponseEntity<Void> pauseDeployment(@PathVariable UUID id);

    // 14. Resume a paused deployment
    @PutMapping("/{id}/resume")
    public ResponseEntity<Void> resumeDeployment(@PathVariable UUID id);
```

```
    // 15. Check readiness of target environment before deploying
    @GetMapping("/check-readiness")
    public ResponseEntity<ReadinessDTO> checkReadiness(@RequestParam String targetEnv);
}
```

Why REST? Standardized standard verbs (POST for changing state, GET for retrieval) enables seamless API Gateway routing and standard client SDK generation.

4️⃣ DEEP DIVE: WORKFLOW EXECUTION SERVICE

Execution Lifecycle

1. **Validation**: Check inputs against `RequiredInputDTO`.
2. **Resolution**: Replace Vault references and secrets in the `FlowJson`.
3. **Execution**: The `WorkflowAsyncExecutor` parses the AST (Abstract Syntax Tree) of the flow.
4. **Step Handling**: Parallel/Sequential nodes are evaluated using `CompletableFuture`.

A) Entities

- `WorkflowExecutions`: Tracks `executionId`, `status`, `logs`.
- `FlowJson`: Document DB storage for the dynamic DAG definition.
- `WorkflowExecutionNodes`: Tracks individual step statuses (`StepStatus` enum: `PENDING, RUNNING, SUCCESS, FAILED`).

B) Repositories

- MongoRepositories for dynamic schemas (chat history, step variable contexts). Pagination is strictly enforced via `Pageable`.

C) Services

- `initiateWorkflowExecution()`: Starts boundary.
- `continueConversationStream()`: Handles Server-Sent Events (SSE).
- `validateAndExecuteWorkflowAsync()`: Runs the graph parsing logic.

D) Controllers (15 APIs)

```
@RestController
@RequestMapping("/api/workflows")
public class WorkflowExecutionController {

    // 1. Create a new workflow definition draft
    @PostMapping
    public ResponseEntity<WorkflowDTO> createWorkflow(@RequestBody WorkflowCreateDTO dto);

    // 2. Fetch workflow metadata
    @GetMapping("/{id}")
    public ResponseEntity<WorkflowDTO> getWorkflow(@PathVariable String id);

    // 3. Start a new execution context
    @PostMapping("/{id}/execute")
    public ResponseEntity<ExecutionResponseDTO> executeWorkflow(@PathVariable String id,
@RequestBody Map<String,Object> inputs);
```

```java
    // 4. Get active execution status
    @GetMapping("/executions/{id}")
    public ResponseEntity<ExecutionDetailsDTO> getExecution(@PathVariable UUID id);

    // 5. Retry a workflow from the last failed node
    @PostMapping("/executions/{id}/retry")
    public ResponseEntity<ExecutionResponseDTO> retryExecution(@PathVariable UUID id);

    // 6. Rollback state to a previous node
    @PostMapping("/executions/{id}/rollback")
    public ResponseEntity<Void> rollbackExecution(@PathVariable UUID id);

    // 7. Find executions by state (e.g. FAILED)
    @GetMapping("/executions/status/{status}")
    public ResponseEntity<List<ExecutionDetailsDTO>> getExecutionsByStatus(@PathVariable
String status);

    // 8. List workflows per project
    @GetMapping("/project/{projectId}")
    public ResponseEntity<List<WorkflowDTO>> getWorkflowsByProject(@PathVariable UUID
projectId);

    // 9. Validate a DAG for cycles/missing vars
    @PostMapping("/{id}/validate")
    public ResponseEntity<ValidationResultDTO> validateWorkflow(@PathVariable String id);

    // 10. Update workflow definition
    @PutMapping("/{id}")
    public ResponseEntity<WorkflowDTO> updateWorkflow(@PathVariable String id, @RequestBody
WorkflowUpdateDTO dto);

    // 11. Archive/Delete workflow
    @DeleteMapping("/{id}")
    public ResponseEntity<Void> deleteWorkflow(@PathVariable String id);

    // 12. Fetch deep execution step logs
    @GetMapping("/executions/logs/{id}")
    public ResponseEntity<List<ExecutionLogDTO>> getExecutionLogs(@PathVariable UUID id);

    // 13. System execution metrics (queue depth, avg time)
    @GetMapping("/executions/metrics")
    public ResponseEntity<MetricsDTO> getExecutionMetrics();

    // 14. Pause a long-running execution
    @PostMapping("/{id}/pause")
    public ResponseEntity<Void> pauseWorkflow(@PathVariable String id);

    // 15. Resume from pause
    @PostMapping("/{id}/resume")
```

```
    public ResponseEntity<Void> resumeWorkflow(@PathVariable String id);
}
```

5️⃣ TOOL / MCP INTEGRATION SERVICES

- **JSON-RPC**: Standard format used to interact with external MCP (Model Context Protocol) servers. It relies heavily on strict DTO mapping.
- **WebClient**: Used exclusively over `RestTemplate` due to its reactive non-blocking capabilities, which is crucial when doing an external API call that might take 30+ seconds.
- **Resilience**: Integrated Resilience4J for **Circuit Breaking** (to short-circuit if a 3rd party tool API is down) and **Timeouts** to prevent thread exhaustion.

6️⃣ DESIGN PATTERNS USED

- **Strategy Pattern**: Used for Workflow Step Handling (e.g., `executeAgentNode`, `executeToolNode`). Allows dynamically swapping execution logic based on node type.
- **Factory Pattern**: `ToolHandlerFactory` orchestrates which Tool Client to construct based on external specification.
- **Template Method Pattern**: Used in abstract Execution Flow where base steps (validate, run, cleanup) are standard, but the `run` method is overridden.
- **Observer Pattern**: Spring `@EventListener` and Kafka topics used heavily to publish `DeploymentStatusEvents` without tight coupling.
- **Circuit Breaker Pattern**: Wraps external tool/MCP calls to prevent runaway cascade failures.
- **Orchestration**: The `WorkflowExecutionService` acts as a central Orchestrator (Saga Orchestration) rather than relying on decentralized choreography, ensuring a single source of truth for execution state.

7️⃣ DATABASE DESIGN

To ensure zero cross-coupling, we adopt **Database-per-Service**:

1. **Deployment DB (PostgreSQL)**
    - Tables: `deployments`, `deployment_tasks`, `deployment_events`.
    - Keys: `id` (UUID) as PK, `workflow_id` as indexed standard string.
    - Strategy: Highly normalized, strict foreign keys for ACID compliance.
2. **Execution DB (MongoDB)**
    - Collections: `workflows`, `executions`, `flow_json`.
    - Keys: `_id` standard Mongo Object IDs, indexed on `executionId`.
    - Strategy: Schema-less nature perfectly maps to dynamic JSON DAG architectures.

8️⃣ COMMUNICATION STYLE

- **Synchronous (REST)**: UI to API Gateway, Gateway to Services.
- **Asynchronous (Event-Driven)**: Kafka is used for `DeploymentCallbacks`, `ExecutionStateUpdates`.
- **Handling Failures**: Dead-letter Queues (DLQ) in Kafka ensure that failed deployment events can be manually analyzed and retried.

9️⃣ CONFIGURATION & DEPENDENCIES

Core Dependencies per Service:

- `spring-boot-starter-webflux` : Non-blocking I/O for Workflows.
- `spring-cloud-starter-circuitbreaker-reactor-resilience4j` : Fault tolerance.
- `spring-kafka` : Event messaging.

General Config (`application.yml`):

```yaml
server:
  port: 8081
spring:
  kafka:
    bootstrap-servers: broker:9092
  datasource:
    url: jdbc:postgresql://db:5432/deployments
resilience4j:
  circuitbreaker:
    instances:
      toolClient:
        waitDurationInOpenState: 10000ms
        slidingWindowSize: 10
```

🔟 DEPLOYMENT ARCHITECTURE

- **Docker**: Each microservice has a multi-stage `Dockerfile` .
- **Kubernetes**: Deployed via Helm charts. Including `Deployment` manifests, `Services` (ClusterIP), and an external `Ingress` mapping to the Gateway.
- **Probes**:
  - `livenessProbe` : `/actuator/health/liveness`
  - `readinessProbe` : `/actuator/health/readiness`
- **Scaling**: HPA (Horizontal Pod Autoscaler) bound to memory/CPU metrics for the Execution Service.

1️⃣ 1️⃣ PRODUCTION READINESS

- **Idempotency**: All `POST` endpoints creating executions use a custom `X-Idempotency-Key` header, stored in Redis to block duplicate charges/runs.
- **Distributed Tracing**: `Micrometer Tracing` with Zipkin/Jaeger to track a request traversing Gateway -> Execution -> Tool -> Deployment.
- **Security**: Stateless JWT validation at the Gateway; RBAC applied downstream via Spring Security `@PreAuthorize` .
- **Performance**: Heavy database queries leverage Redis `@Cacheable` for standard lookups (e.g., Workflow definitions are cached, Executions are not).